

What is DevOps?

DevOps = Development + Operations

Traditionally , software development and operations were handled by two separate teams.

1. Dev : Developers write code , implement features and fix bugs
2. Operations : This team deploys applications, manages servers, monitors applications and ensures they run properly.

What was the problem with the traditional approach ?

Imagine you're developing a website for an online shopping platform.

1. Developers write the code on their computer.
2. They give the completed code to the operations team
3. The operations team tries to deploy it on a server
4. The application crashes because the server has different configurations
5. Developers say the code works on their computers
6. The operations team has to investigate the deployment failure

This creates delays and disagreements between teams

How does DevOps solve this?

DevOps brings development and operations together. Instead of manually transferring code and deploying it, the entire process is automated as much as possible.

For ex:

1. A developer writes code
2. The code is uploaded to Github
3. Jenkins automatically detects the changes
4. Jenkins builds the application
5. Automated tests check whether the code works
6. If the tests pass, the application is deployed
7. Monitoring tools continuously check the application's health

This allows teams to release software faster and more reliably

Why do we need DevOps ?

1. Encourages the collaboration and shared responsibility
2. Automates deployment using the CI/CD pipelines
3. Uses containers such as Docker to provide consistent environments
4. Uses automated testing to identify bug before deployment
5. Uses monitoring and alerting tools.

Ex : Suppose you run an e-commerce website and you want to add a new payment method.

Without DevOps : Developers implement the feature, the operations team manually deploys it , the testing may be incomplete and the deployment could cause downtime.

With DevOps : Developers commit the code, Automated tests run. The application is built and deployed through a pipeline and Monitoring tools check whether the application is working. If something does wrong the team rolls back the deployment if necessary

The DevOps LifeCycle

1. Plan : Decide what needs to be developed. Before writing code , the team needs to understand the requirements.

Suppose you are developing an e-commerce website(cart) .

The team needs to decide:

1. What features should the cart can have?
2. How should the users add and remove the products?
3. How should the cart calculate the total price?
4. How much time will development take?
5. Who is responsible for each task? Like Ui (frontend team), APIs (Backend team), Automated tests(QA) , deployment infra(Operations)

Common tools

- **Jira:** Tracks tasks, bugs, and project progress.
- **Confluence:** Stores project documentation.
- **Trello:** Organizes tasks using boards and cards.
- **Azure Boards:** Manages work items and development tasks.

2. Code : Write and manage the application's source code. Developers now start implementing the features.

For example a backend developer might write a python API to add products to the shopping cart. But what happens when multiple developers work on the same project?

They need a way to :

1. Track code changes
2. Collaborate without overwriting each other's work
3. Revert the unwanted changes
4. Review code before merging it

This is where Git and Github come in.

What is Git ?

Git is a distributed version control system. It tracks changes in source code over time. For example , suppose you have a project called shopping cart

1. We initialize Git using git init
After modifying your code , you can stage and commit the changes (commit is a snapshot of your changes)

What is GitHub ?

GitHub is a platform for hosting Git repos and collaborating on Code. When you push your code to GitHub other developers can review your code, create branches and submit the pull requests.

Git and GitHub are not the same thing. Git is the tool; GitHub is a platform that uses Git.

3. Build Converts source code into an application that can be executed or deployed. After developers write the code, it needs to be built.

A build process can include :

1. Downloading the dependencies
2. Compiling source code
3. Running preliminary checks
4. Packaging the application

Common build tools	
Tool	Purpose
Jenkins	Automates build processes
Maven	Builds Java applications
Gradle	Builds Java and other JVM applications
Docker	Packages applications into container images
GitLab CI/CD	Automates building, testing, and deployment

4. Test : Verify that the application works correctly before releasing it.

There are several types of testing

1. Unit testing : Tests individual functions or components.
2. Integration testing : Tests whether different components work together
3. Performance testing : Checks how the application performs under different workloads
4. Security testing : checks for security vulnerabilities.

Why is automated testing important?

Imagine manually testing your entire website every time a developer changes one line of code. That would be time - consuming .

Instead you can configure a CI/CD pipeline to automatically execute tests whenever new code is committed. **If the test fails, the pipeline can stop before deployment.** This helps catch bugs early.

5. Release : Prepare an approved version of the application for deployment.

After the application passes the required tests, the team prepares it for the release.

A release may involve:

1. Selecting the version to deploy
2. Tagging the code
3. Approving the release
4. Preparing the deployment configurations
5. Planning how to roll back if necessary

Release vs Deploy

Release: Making a particular version approved and ready for users.

Deployment: Installing or updating that version in an environment.

6. Deploy : Make the application available in the target environment.
Once the application is approved , it needs to be deployed

Deployments can introduce problems:

1. The application may crash
2. The server may run out of memory
3. Users may experience downtime
4. The new version may contain bugs.

DevOps uses automated deployment strategies to reduce these risks.

- 1. Rolling deployment:** In a rolling deployment, the application instances are updated gradually.

Suppose your application has 4 instances:

Instance1, Instance2, Instance3, Instance4

Instead of replacing all 4 simultaneously, you update them one at a time or in small groups. This allows some instances to continue serving users while others are updated.

- 2. Blue Green deployment : This allows some instances to continue serving users while others are updated.** In blue - green deployment , you maintain 2 environments

1. Blue : The currently active version
2. Green : The new version

You deploy the new version to green and test it . Once it is ready , you switch traffic from blue to green.

If the new version has a problem, you can switch traffic back to blue, provided the old environment remains available and compatible.

- 3. Canary deployment:** In a canary deployment, you initially release the new version to a small percentage of users. If the new version works correctly, you gradually increase the traffic . If problems occur, you can stop the rollout

7. Operate and Monitor: Maintain the application and monitor its performance after deployment.

Your shopping website is now live but deploying the application doesn't mean the work is finished you need to ensure that

The website is available , the application responds quickly , the database is working , the application isn't consuming excessive resources.

What is monitoring ? Monitoring means continuously collecting and analyzing information about an application's health and performance.

How does the feedback loop work? Suppose your website becomes slow after deployment. Prometheus detects increased response times, Grafana displays the performance degradation, An alert notifies the team , Developers investigate the logs , They identify and fix the problem , The fix goes through the DevOps pipeline again.

Step	Tool	What happens
1. Code	Git	Developers track changes locally.
2. Upload	GitHub	Developers push code to a remote repository.
3. CI	Jenkins	Jenkins detects code changes and starts a pipeline.
4. Build	Docker	Jenkins builds a Docker image.
5. Test	Pytest	Automated tests verify the application.
6. Store	Container registry	The Docker image is stored for deployment.
7. Deploy	Kubernetes	Kubernetes runs the application containers.
8. Configure	Ansible	Servers and other infrastructure can be configured automatically.
9. Monitor	Prometheus	Metrics are collected from the application.
10. Visualize	Grafana	Metrics are displayed in dashboards.

Tools	Main difference
Git vs. GitHub	Git tracks code changes; GitHub hosts repositories and supports collaboration.
Jenkins vs. Docker	Jenkins automates workflows; Docker packages and runs applications in containers.
Docker vs. Kubernetes	Docker builds and runs containers; Kubernetes manages containerized applications across a cluster.
Ansible vs. Terraform	Ansible automates configuration and tasks; Terraform provisions and manages infrastructure.
Prometheus vs. Grafana	Prometheus collects and stores metrics; Grafana visualizes metrics.
CI vs. CD	CI integrates and tests code; CD prepares or automatically deploys changes.
Build vs. Deploy	Build creates an artifact; deployment makes the application available in an environment.
Release vs. Deploy	Release makes a version available for use; deployment installs it in an environment.

7 Cs of DevOps :

1. Continuous Development : Continuously developing software by adding new features and improving existing ones in small increments . Traditionally , developers might spend months building an entire application before releasing it but now developers divide the application into smaller features and develop them incrementally.

- a. Software is developed in small, manageable increments
- b. Developers frequently update the code
- c. Version control systems such as Git help manage changes(Git helps identify and revert the problematic changes)
- d. Smaller changes make bugs easier to identify and fix

2. Continuous Integration : Continuously merging the developer's code changes into a shared repository and automatically building and checking the application.All developers work on different parts of the same application. When they finish their work, their code needs to be integrated. If they wait several weeks before combining their code, conflicts and integration problems may occur. Continuous Integration solves this problem by encouraging frequent integration.

Suppose developer 1 finishes the login feature

- a. Step1 : meena pushes her code to github
- b. Jenkins detects the new code
- c. Jenkins automatically starts the CI pipeline
- d. Jenkins builds the application and runs the configured checks
- e. If everything passes , the build is marked as successful if build fails developers are notified so they can fix the problem

Continuous Integration focuses on frequently integrating code and automatically building and checking it.

3. Continuous Testing : Automatically testing the application throughout the development process to detect bugs as early as possible. Even if the application builds successfully, it may still contain bugs. For example, your food delivery application may compile successfully, but the payment feature might calculate the wrong amount. This is why testing is necessary
Why is testing important?

Suppose you modify the payment feature. The change might accidentally break the checkout process. Automated tests can detect this problem before the updated application reaches users. This reduces the risk of introducing bugs into production

Continuous Testing ensures that changes are tested regularly and that new features don't break existing functionality.

4. Continuous Delivery and Continuous Deployment:

- a. **Continuous Delivery :** Automatically building, testing, and preparing software so that it is always ready for deployment, with production deployment typically requiring manual approval.
- b. **Continuous Development :** Automatically deploying every code change that passes the required checks to production, without requiring manual approval for each deployment.

5. Continuous Monitoring : Continuously observing the application's performance, availability, and health after deployment.

Once your food delivery application is deployed, you need to ensure that it works correctly. Imagine that 10,000 users are ordering food during dinner time. Suddenly, the application becomes slow. Without monitoring, the team might not notice the problem until users start complaining. Continuous Monitoring helps detect such problems early.

How do we monitor? (There are 3 important types of monitoring data)

1. Metrics are numerical measurements of application performance. Ex : CPU utilization, Memory usage , Response time, Request rate, Error rate
For example, if CPU usage reaches 95%, the application might be under heavy load.
2. Logs are records of events generated by an application. Logs help developers investigate errors and understand what happened.
3. Distributed traces show how a request travels through different services.
For example, when a user places an order, the request might pass through: API Gateway , Order Service , Payment Service , Database so a distributed trace helps identify which component is causing a delay.

How do Prometheus and Grafana help?

Prometheus collects and stores metrics.

Grafana displays those metrics in dashboards.

For example:

Suppose your application becomes slow during dinner time.

Prometheus collects CPU and response-time metrics.

An alert can notify the DevOps team when a configured threshold is exceeded.

Grafana can display a dashboard showing CPU usage and response times, helping the team investigate the issue.

Continuous Monitoring focuses on detecting performance problems, errors, and availability issues in the running application.

6. Continuous Feedback : Continuously collecting feedback from users, monitoring systems, and development teams to improve the application. Monitoring tells you how your application is performing while Feedback helps you understand what needs to be improved.

Ex : Suppose users report that the food delivery application takes too long to display restaurant search results. The team follows this process:

1. Collect user complaints and application performance data
2. Analyze the feedback and identify the problem
3. Discover the search API takes too long to respond
4. Optimize the database queries and improve the search functionality
5. Release the updated application
6. Monitor whether response times improve and complaints decrease

- 7. Continuous Operations** Continuously maintaining and operating the application to ensure availability, reliability, and smooth functioning. Continuous Operations focuses on minimizing downtime and maintaining the application.

What activities are involved ?

- a. **High Availability** : High availability means designing systems so that they remain accessible even when some components fail. For example, instead of running your application on only one server, you can run multiple instances.
- b. **Load Balancing** : A load balancer distributes incoming requests across multiple servers. If one server becomes unavailable, the load balancer can stop sending traffic to it, depending on its health-check configuration.
- c. **Zero-Downtime Deployment** : Using deployment strategies such as Rolling deployment, Blue-green deployment, Canary deployment
- d. **BackUp and recovery** : The team needs to protect important data and prepare for failures. For example, regular database backups help recover data if a serious failure occurs.

Continuous Operations focuses on maintaining the application, handling failures, and ensuring reliable service.

No.	Stage	What happens in the food delivery application?
1	Continuous Development	Developers continuously implement features such as login and order tracking.
2	Continuous Integration	Developers frequently merge their code, and automated builds and checks run.
3	Continuous Testing	Automated tests verify that new features work and existing features aren't broken.
4	Continuous Delivery/Deployment	Tested code is prepared for release or automatically deployed to production.
5	Continuous Monitoring	The team monitors server health, response times, and errors.
6	Continuous Feedback	User complaints and system insights are used to identify improvements.
7	Continuous Operations	The team maintains availability, handles failures, and performs updates.